

Report (Lab 1 - 4) STT

Chavda Haarit Ravindrakumar (23110077)

Github Link : https://github.com/haarit19058/stt_labs

Lab 01

Introduction and Concepts

Lab 01 aims to teach version control systems (VCS). The fundamentals of these tools, which are essential for effective management and maintaining the project development workflows, will be covered in this lab. Additionally, the lab discusses GitHub Actions, a full-proof framework that makes automated Continuous Integration (CI) and Continuous Deployment (CD) procedures possible.

Concepts :

Version Control System (VCS): What is it?

A version control system is a tool that facilitates collaborative work on a project by tracking file changes over time and making revision management simple for multiple users. When multiple contributors edit the duplicate files, developers can compare changes, return to earlier iterations, and settle disputes.

CI/CD: What is it?

Continuous Integration (CI) ensures frequent integrations and early error detection by automatically building and testing code whenever changes are committed. The automated process of swiftly and securely delivering code changes to production, known as continuous deployment (or CD), minimizes the need for human intervention when releasing updates and new features.

Tools and Setup

I have Manjaro Linux as an operating system, and by default, Git was installed. Git can be installed on Manjaro by running "sudo pacman -S git base-devel" if it hasn't already been installed.

The Pylint GitHub Action sets up continuous integration (CI) and continuous deployment (CD). As part of the CI/CD process on GitHub, this tool automatically evaluates the quality of Python code to ensure standards are upheld and possible errors are identified early.

Methodology and Execution

The initial step involves configuring Git with user credentials. The following commands are used to set the global username and email:

```
~ > git config --global user.name "Haarit"  
~ >  
~ > git config --global user.email "haarit19058@gmail.com"  
~ > █
```

We must tell git whether the directory is safe; therefore, git maintains a list of safe directories. We can omit that part by piping the central command into `grep -v safe. Directories`. To verify the configuration, use:

```
~ > git config --list | grep -v safe.directory  
  
user.email=haarit19058@gmail.com  
user.name=Haarit  
credential.helper=store  
credential.credentialstore=plaintext  
credential.https://dev.azure.com.usehttppath=true
```

Next, a new project directory called `lab1` is created, and navigate to that folder:

```
~ > mkdir Lab1  
~ > cd Lab1  
~/Lab1 > █
```

The Git repository is then initialized in the folder, and a README file is created with a brief description:

```
~/Lab1 > git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
hint:
hint: Disable this message with "git config set advice.defaultBranchName false"
Initialized empty Git repository in /home/haarit/Lab1/.git/
~/Lab1 master > ls
~/Lab1 master > echo "This is the first file for the amazing lab1" > README.md
~/Lab1 master ?1 > ls
README.md
~/Lab1 master ?1 > cat README.md
This is the first file for the amazing lab1
~/Lab1 master ?1 > █
```

The README file is added to the staging area by adding all files or just the README. Specifically:

```
This is the first file for the amazing lab1
~/Lab1 master ?1 > git add .
~/Lab1 master +1 > █
```

After staging, the first commit is made with an appropriate message:
Git commit -m "First commit of the course"

```
~/Lab1 master +1 > git commit -m "First commit of the course"
[master (root-commit) d462f2b] First commit of the course
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
~/Lab1 master > █
```

The history of commits can be viewed in git log.

```
commit d462f2b55a72671ee358a0b88b337b04fe8333dd (HEAD -> master)
Author: Haarit <haarit19058@gmail.com>
Date:   Fri Aug 29 16:59:56 2025 +0530

    First commit of the course
(END)
```

Create a remote repository by following the steps shown in the figures.

Go to github.com, sign in with your email ID, and click on the green button that says "new." Click that button to create a new repository.

The screenshot shows the GitHub 'Create a new repository' page. It is divided into two main sections: 'General' and 'Configuration'.

General Section:

- Owner *:** A dropdown menu showing 'haarit19058'.
- Repository name *:** A text input field containing 'stt_lab01_test'. Below it, a green checkmark indicates 'stt_lab01_test is available.'
- Description:** A large text area for the repository description, currently empty. Below it, it says '0 / 350 characters'.

Configuration Section:

- Choose visibility *:** A dropdown menu set to 'Public'.
- Add README:** A section with a toggle switch set to 'Off'. Below it, it says 'READMEs can be used as longer descriptions. [About READMEs](#)'.
- Add .gitignore:** A dropdown menu set to 'No .gitignore'. Below it, it says '.gitignore tells git which files not to track. [About ignoring files](#)'.
- Add license:** A dropdown menu set to 'No license'. Below it, it says 'Licenses explain how others can use your code. [About licenses](#)'.

At the bottom right of the form is a green button labeled 'Create repository'.

A remote repository named STT is created on GitHub to connect the local repository to GitHub. The current branch is renamed to main for consistency with GitHub standards, and the remote repository is linked:

GitHub uses the main branch as its main branch, and git uses master as the default branch, so we need to change the name of the default branch from master to main using: `git branch -M main`

Now add the remote repo URL and link it with the local repository:

Use `git remote -v` to see the remote repository link

```
~/Lab1 master > git branch -M main
~/Lab1 main > git remote add origin https://github.com/haarit19058/stt_lab01_test.git
~/Lab1 main > git remote -v
origin https://github.com/haarit19058/stt_lab01_test.git (fetch)
origin https://github.com/haarit19058/stt_lab01_test.git (push)
```

Push the commits to the remote branch and set the default upstream using `git push --set-upstream origin main`.

```
~/Lab1 main > git push --set-upstream origin main

Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 259 bytes | 259.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/haarit19058/stt_lab01_test.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Cloning an existing repository :

```
~/Rubbish > git clone https://github.com/haarit19058/BuildingOS.git
Cloning into 'BuildingOS'...
remote: Enumerating objects: 2207, done.
remote: Counting objects: 100% (227/227), done.
remote: Compressing objects: 100% (148/148), done.
remote: Total 2207 (delta 75), reused 184 (delta 62), pack-reused 1980 (from 1)
Receiving objects: 100% (2207/2207), 154.19 MiB | 12.90 MiB/s, done.
Resolving deltas: 100% (819/819), done.
~/Rubbish > █
```

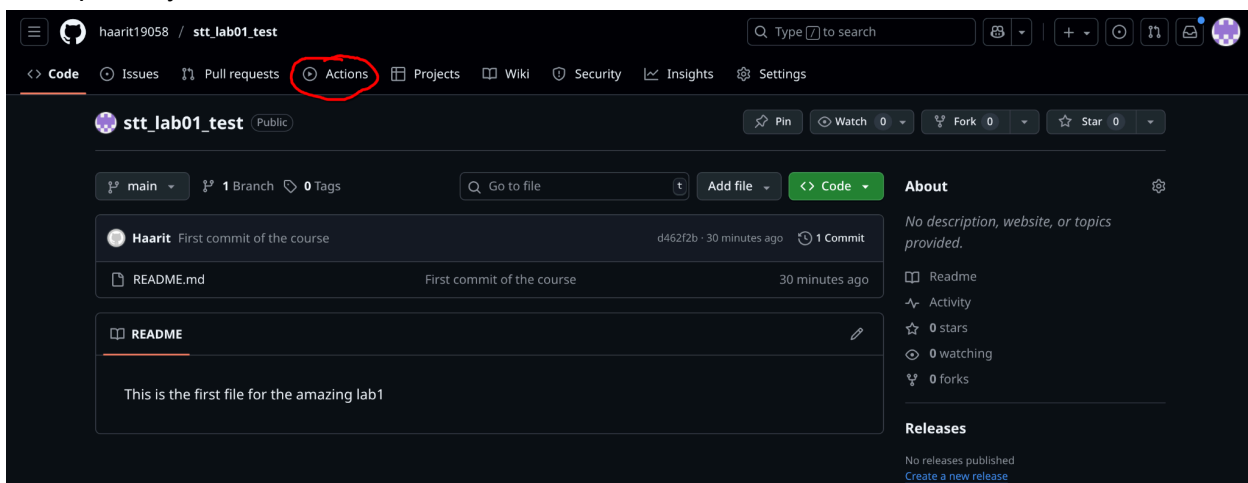
To get the code from the remote repo, execute this command: `git pull origin main`.

```
~/Rubbish/BuildingOS main > git pull
remote: Enumerating objects: 4, done.
remote: Counting objects: 100% (4/4), done.
remote: Compressing objects: 100% (1/1), done.
remote: Total 3 (delta 1), reused 3 (delta 1), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 264 bytes | 264.00 KiB/s, done.
From https://github.com/haarit19058/BuildingOS
   d339b13..553490f  main       -> origin/main
Updating d339b13..553490f
Fast-forward
 pull.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 pull.md
```

Now that Lab 01 has reached its midway point, it's time to set up GitHub Actions—one of the most exciting and productive features of GitHub. We'll integrate Pylint with GitHub Actions to automatically check code quality on every push or pull request.

Pylint is a static code analysis tool for Python that examines source code without executing it. It helps developers identify errors, enforce coding standards based on PEP 8, and provides suggestions for refactoring and improving code consistency. Pylint is ideal for keeping code reliable, readable, and maintainable throughout a project's lifecycle.

For any workflow on GitHub, we need to add a YAML file corresponding to that particular action in `\.github/workflow/actionname.yaml`. Using the GU, as shown below, we will add pylint to our repository.



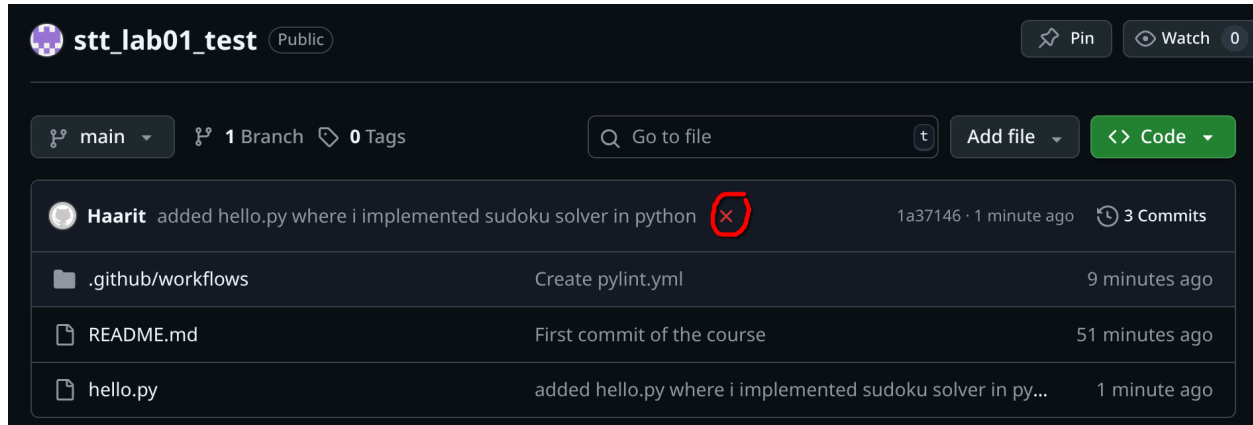
The top screenshot shows the 'Get started with GitHub Actions' page. It features a search bar with 'pylint' entered, showing 'Found 1 workflow'. The workflow is 'Pylint' by GitHub Actions, with a 'Configure' button and a 'Python' language indicator.

The bottom screenshot shows a GitHub repository interface for 'stt_lab01_test'. The 'Actions' tab is selected, displaying the 'pylint.yml' workflow file. The workflow is configured to run on 'ubuntu-latest' and uses a matrix for Python versions: '3.8', '3.9', and '3.10'. The steps include checking out the code, setting up Python, installing dependencies, and running 'pylint' on the files. A red arrow points to the 'Commit changes...' button in the top right corner of the workflow editor.

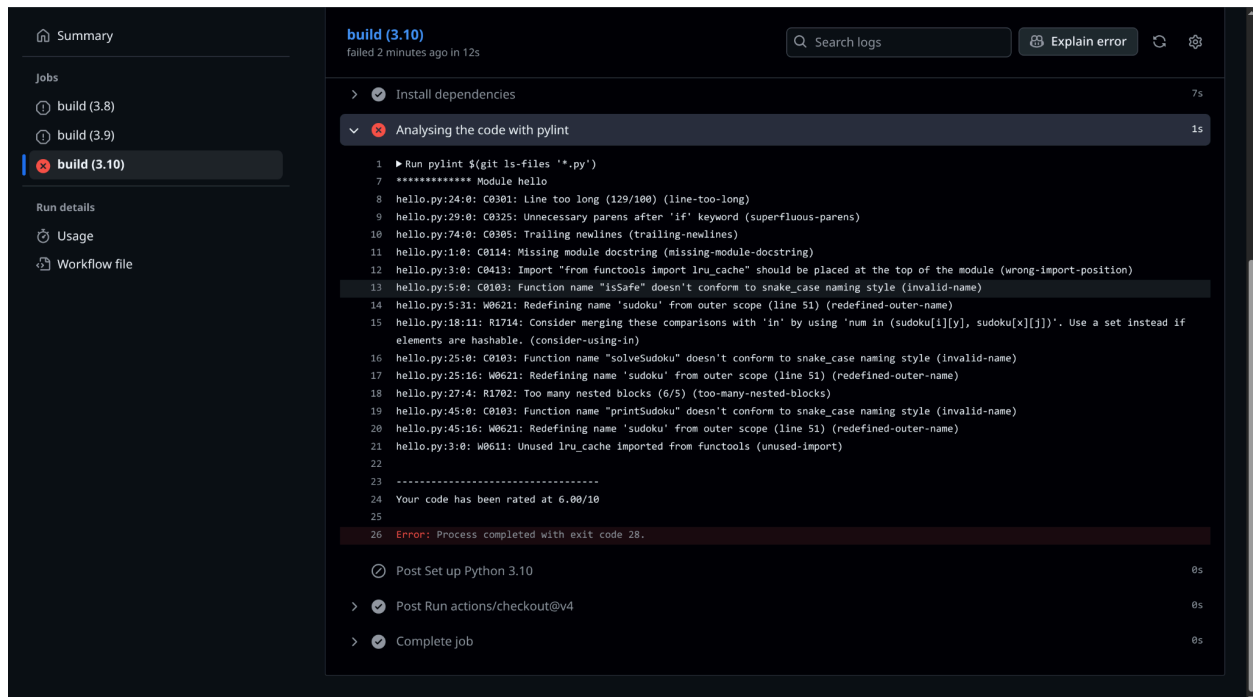
```
1 name: Pylint
2
3 on: [push]
4
5 jobs:
6   build:
7     runs-on: ubuntu-latest
8     strategy:
9       matrix:
10        python-version: ["3.8", "3.9", "3.10"]
11     steps:
12       - uses: actions/checkout@v4
13       - name: Set up Python ${{ matrix.python-version }}
14         uses: actions/setup-python@v3
15         with:
16           python-version: ${{ matrix.python-version }}
17       - name: Install dependencies
18         run: |
19           python -m pip install --upgrade pip
20           pip install pylint
21       - name: Analysing the code with pylint
22         run: |
23           pylint $(git ls-files '*.py')
```

Result Analysis and Discussion

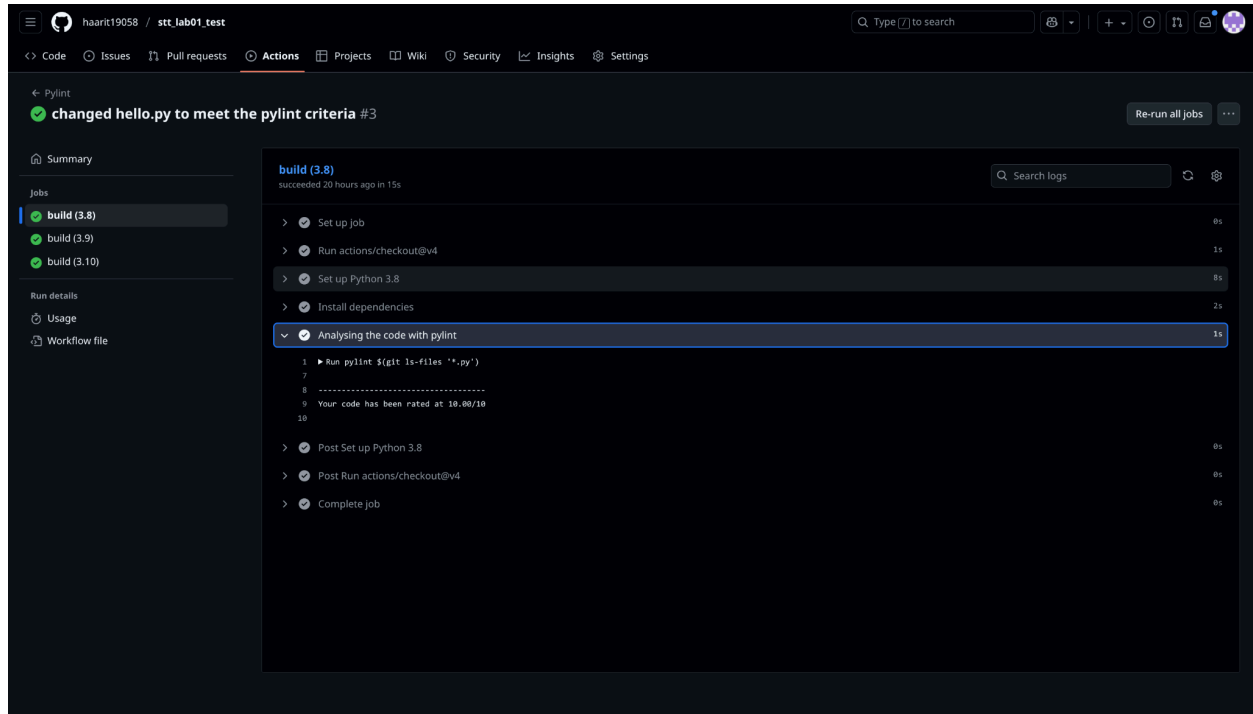
I have added hello.py, where I implemented a Sudoku solver in Python and pushed the changes to the remote repo. It automatically runs the pylint, and this is the output ..



pylint runs many checks on the code. Here is the analysis for this commit ..



Now, I have fixed the hello.py by removing multiple nested for loops and adding comments for functions, and the overall score given by pylint has increased.



Conclusion

Overall, this lab highlighted the importance of version control and automation in modern software workflows. The skills learned here will enable developers to manage projects more effectively, maintain high-quality codebases, and streamline development processes through continuous integration and deployment practices. These practices are essential for professional software development and collaboration in academic and industry settings.

References

1. Lecture Slides
2. Github cheatSheet [\(link\)](#)

Lab 02

Introduction and Concepts

This lab aims to teach the fundamentals of mining open source software (OSS) repositories. It involves processing and analyzing commits from the GitHub version control system for widely used real-world projects. In practice, the developers contributing to those projects best determine the definition of a bug and its fix. Existing tools may struggle to capture project-specific or domain-specific semantics. The primary goal of this lab assignment is to develop a framework for understanding how developers perceive bug-fixing commits.

Tools and Setup

1. Torch: A framework for running machine learning models efficiently.
2. Transformers: A library for working with large language models (LLMs) and natural language processing tasks.
3. PyDriller: A tool that traverses GitHub repositories commit by commit and provides a convenient API for mining software repository data.
4. Pandas: A library for handling and analyzing data that helps work with CSV files.
5. Anaconda: A tool for managing Python environments and packages.
6. SEART: search engine for GitHub repositories

Creating a new environment and installing necessary packages :

- `conda create --name pystt python=3.12`
- `Pip install torch transformers pydriller pandas.`

Methodology and Execution

First, we must select a repository we will use throughout the assignment. For that, we will use SEART to search for GitHub repos. I put the following criteria :

- `stars >= 10k`
- `commits >= 1000`
- `language == python`
- The project must be well-known and used by many.y

SEART Statistics Publication About

General

Search by keyword in name Contains python

License Has topic Uses Label

History and Activity

Number of Commits 1000 max min max

Number of Contributors min max

Number of Issues min max

Number of Pull Requests min max

Number of Branches min max

Number of Releases min max

Popularity Filters

Number of Stars 10000 max

Number of Watchers min max

Number of Forks min max

Size of codebase ⓘ

Non Blank Lines min max

Code Lines min max

Comment Lines min max

Date-based Filters

Created Between dd/mm/yyyy dd/mm/yyyy

Last Commit Between dd/mm/yyyy dd/mm/yyyy

Additional Filters

Sorting Name Ascending

Repository Characteristics

☐ Exclude Forks ☐ Has License

☐ Only Forks ☐ Has Open Issues

☐ Has Wiki ☐ Has Pull Requests

Search

So from the result, I read through a few of the repos and finally decided to work on agno. Agno is a full-stack framework for building Multi-Agent Systems with memory, knowledge, and reasoning. Here is the link to the repository: <https://github.com/agno-agi/agno.git>.

Now moving on to the next part, we need to identify which commits are bug-fixing commits. For that, I use the straightforward approach of checking whether the developer's commit message contains certain words like "fix" or "bug". If the message includes such words, classify it as a bug-fixing commit and store the data in the table.

Here are the words that I took from the lecture slides to identify bug fix commits :

```
bug_keywords = ["fixed", "bug", "fixes", "fix", "crash", "solves",
               "resolves", "issue", "regression", "fall back",
               "assertion", "coverity", "reproducible", "stack-wanted",
               "steps-wanted", "testcase", "failure", "fail", "npe",
               "except", "broken", "differential testing", "error",
               "hang", "test fix", "steps to reproduce", "leak",
               "stack trace", "heap overflow", "freez", "problem",
               "overflow", "avoid", "workaround", "break", "stop"]
```

This is the function to check whether a message is a bug-fix message or not.

```
def is_bug_commit_naive(commit):  
    message = commit.msg  
    msg_lower = message.lower()  
    return any(kw in msg_lower for kw in bug_keywords)
```

Now we use the API provided by PyDriller to traverse the commit and store the necessary information in a list if the commit is a bug-fixing commit.

```
commit_data = []  
  
for commit in Repository(repo_url).traverse_commits():  
    if is_bug_commit_naive(commit):  
        commit_data.append({  
            "hash": commit.hash,  
            "message": commit.msg,  
            "parents": commit.parents,  
            "is_merge": commit.merge,  
            "files_modified": [mod.filename for mod in commit.modified_files]  
        })
```

Now we will store the data in a CSV file as required for the assignment.

```
df = pd.DataFrame(commit_data)  
csv_path = "potential_bug_fix_commits1.csv"  
df.to_csv(csv_path, index=False)  
  
print(f"\nSaved {len(df)} potential bug-fixing commits to {csv_path}")
```

Now, we move on to the diff extraction and analysis part. As mentioned in the assignment, we will use the LLM to generate tags on a file basis. First, we will load the model and then add a new field in the previous code for LLM inference.

The function is to make inferences and set up the model.

```

# Load model once
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
tokenizer = AutoTokenizer.from_pretrained("mamiksik/CommitPredictorT5")
model = AutoModelForSeq2SeqLM.from_pretrained("mamiksik/CommitPredictorT5").to(device)
model.eval()

MAX_INPUT_TOKENS = 512 # Limit input size to prevent OOM
MAX_OUTPUT_TOKENS = 128 # Limit generated text length

def safe_infer(diff_text):
    """Run model inference safely using only file.diff."""
    if not diff_text:
        return ""

    # Tokenize & truncate
    inputs = tokenizer(
        diff_text,
        return_tensors='pt',
        max_length=MAX_INPUT_TOKENS,
        truncation=True
    ).to(device)

    with torch.no_grad():
        outputs = model.generate(**inputs, max_length=MAX_OUTPUT_TOKENS)

    return tokenizer.decode(outputs[0], skip_special_tokens=True)

```

The main traversal will use PyDriller to create the dataset per the assignment task.

```

def collect_commit_data(repo_url):
    iter = 1000000000
    commit_data = []

    for commit in Repository(repo_url).traverse_commits():
        if iter == 0:
            return commit_data
        iter -= 1
        if is_bug_commit_naive(commit):
            for file in commit.modified_files:
                try:
                    inference = safe_infer(file.diff)

                    commit_data.append({
                        "hash": commit.hash,
                        "filename": file.filename,
                        "diff": file.diff or "",
                        "llm_inference": inference,
                        "rectified_msg": ""
                    })

                except Exception as e:
                    print(f"Skipped file {file.filename} in commit {commit.hash}: {e}")

            # Free memory
            gc.collect()
            torch.cuda.empty_cache()

    return commit_data

```

Rectifier Formulation:

This is an open-ended task, so there are many ways to make a rectifier. My approach is as follows:

- First, collect the per-file tags by the given LLMS
- use llama-3-8b model using groq api
- Make a prompt that gives data as a file: llm tag and ask Llama to create a commit message based on this data.

I also generated a commit message by passing the file diffs to the model, but it reached the token limit, and due to resource constraints, the next best option I found feasible is to give the file and llm_tags to the model.

Function to get a response from the llama given the data

```
def groq_commit_message(prompt):  
    """Use Groq API to get commit-level rectified message with size check."""  
    prompt = safe_truncate(prompt, max_chars=4000) # ~4000 chars ≈ 1500-2000 tokens  
  
    chat_completion = groq_client.chat.completions.create(  
        model="llama3-8b-8192",  
        messages=[  
            {  
                "role": "system",  
                "content": "You are an expert software engineer skilled at writing precise commit messages"  
            },  
            {  
                "role": "user",  
                "content": prompt  
            }  
        ],  
        temperature=0.3,  
        max_tokens=150  
    )  
    return chat_completion.choices[0].message.content.strip()
```

Now we will integrate this into the previous code to make a final file containing the data as a single table, as asked in the assignment.

```

def collect_commit_data(repo_url, limit=100):
    commit_data = []

    for commit in Repository(repo_url).traverse_commits():
        if limit == 0:
            break
        limit -= 1

        if is_bug_commit_naive(commit):
            file_tags = []

            for file in commit.modified_files:
                try:
                    llm_tag = infer(file.diff or "")
                    file_tags.append(f"{file.filename}: {llm_tag}")
                except Exception as e:
                    print(f"Skipped file {file.filename} in commit {commit.hash}: {e}")

            gc.collect()
            torch.cuda.empty_cache()

            combined_prompt = (
                f"Original commit message:\n{safe_truncate(commit.msg, 500)}\n\n" +
                f"LLM file-level tags (truncated):\n" +
                safe_truncate("\n".join(file_tags), 3000) +
                "\nTask: Write a concise, accurate commit message summarizing all changes. Respond in a single line only commit message."
            )

            try:
                rectified_commit_msg = groq_commit_message(combined_prompt)
            except Exception as e:
                rectified_commit_msg = f"[ERROR: {e}]"

            commit_data.append({
                "hash": commit.hash,
                "developer_msg": commit.msg,
                "rectified_commit_msg": rectified_commit_msg
            })

    return commit_data

```

We will use code bert for the research question to check the similarity between the diff and the developer/llm / rectified messages. We will first load the model, prepare the tokenizer for CodeBER, T, and create a function to get the embeddings for given text. We have stored LLM inference on a per-file basis for a given commit, a list of strings. The normalize text function will concatenate those strings to make a compound tag for the whole commit. More discussion on the result and approach is written under the result and analysis section, and here is just the method and execution.


```

from transformers import RobertaTokenizer, RobertaModel
import torch
import torch.nn.functional as F

# Load CodeBERT
tokenizer = RobertaTokenizer.from_pretrained("microsoft/codebert-base")
model = RobertaModel.from_pretrained("microsoft/codebert-base")
✓ 14.2s

/home/haarit/miniconda3/envs/pystt/lib/python3.13/site-packages/tqdm/auto.py:2
from .autonotebook import tqdm as notebook_tqdm

def get_embedding(text):
    tokens = tokenizer(text, return_tensors='pt', truncation=True, max_length=512)
    with torch.no_grad():
        output = model(**tokens)
    # Use [CLS] token embedding
    return output.last_hidden_state[:,0,:]

def similarity(emb1, emb2):
    return F.cosine_similarity(emb1, emb2).item()

def normalize_text(val):
    if isinstance(val, list):
        return " ".join(str(x) for x in val)
    elif isinstance(val, str):
        return val
    else:
        return ""
✓ 0.0s

```

Evaluating the developer message

```

Developer Eval

import pandas as pd

df = pd.read_csv("master_commits.csv")

total = 0
hits = 0

for _, row in df.iterrows():
    inf_text = normalize_text(row["diff_y"])
    msg_text = normalize_text(row["developer_msg"])

    # if not inf_text or not msg_text:
    #     continue

    sim = similarity(get_embedding(inf_text), get_embedding(msg_text))

    if sim > 0.95:
        hits += 1

    total += 1

print(f"Total: {total}, Hits: {hits}, Accuracy: {hits/total*100:.2f}%")
✓ 1m 2.1s

Total: 335, Hits: 130, Accuracy: 38.81%

```

Evaluating the LLM inference

```

LLM Eval

import pandas as pd

df = pd.read_csv("master_commits.csv")

def normalize_text(val):
    if isinstance(val, list):
        return " ".join(str(x) for x in val)
    elif isinstance(val, str):
        return val
    else:
        return ""

total = 0
hits = 0

for _, row in df.iterrows():
    inf_text = normalize_text(row["diff_y"])
    msg_text = normalize_text(row["llm_inference"])

    # if not inf_text or not msg_text:
    #     continue

    sim = similarity(get_embedding(inf_text), get_embedding(msg_text))

    if sim > 0.95:
        hits += 1

    total += 1

print(f"Total: {total}, Hits: {hits}, Accuracy: {hits/total*100:.2f}%")

```

✓ 1m 4.2s

Total: 335, Hits: 142, Accuracy: 42.39%

Evaluating the rectifier

```

Rectifier Eval

import pandas as pd

df = pd.read_csv("master_commits.csv")

def normalize_text(val):
    if isinstance(val, list):
        return " ".join(str(x) for x in val)
    elif isinstance(val, str):
        return val
    else:
        return ""

total = 0
hits = 0

for _, row in df.iterrows():
    inf_text = normalize_text(row["diff_y"])
    msg_text = normalize_text(row["rectified_commit_msg"])
    # print(row["rectified_commit_msg"])
    # print(msg_text)

    # if not inf_text or not msg_text:
    #     continue

    sim = similarity(get_embedding(inf_text), get_embedding(msg_text))

    if sim > 0.95:
        hits += 1

    total += 1

print(f"Total: {total}, Hits: {hits}, Accuracy: {hits/total*100:.2f}%")

```

✓ 1m 0.9s

Total: 335, Hits: 217, Accuracy: 64.78%

Result Analysis and Discussion

We have our final data as a CSV with columns per the requirement. Out of 1200 commits of the repo, 335 commits are bug fix commits, so in the first part, we store the information of modified files and bug fix commits in the CSV file. Next, we store the LLM inference per-file in the CSV file. We use a llama 3 8b as our rectifier, and using zero-shot prompts, we generate our rectified commit messages and store them in the CSV files.

Research Questions

1. Did the developer write precise commit messages ??

To find an answer to this question, we must define what is precise and to what extent we can tolerate the precision. Do we need detailed commit messages or short tag-like commit messages ?? This choice relies entirely on the project maintainer. But for now, we select how relevant the message is to commit as our criterion, then one option is to use semantic similarity between the commit diff and the message written by the developer. However, using CodeBert or any similar model would give a very low score, as one text is code and another is English. We need to train a model that maps the code's meaning to human language. Fortunately, our requirements are satisfied by a pre-existing model codeBERT by Microsoft that maps code to Natural Language. We will keep a similarity threshold to determine how good the message is, and if it passes, then we will classify it as a hit.

We keep a very high similarity threshold to get precise results on the relevant message related to the commit. Therefore, I have set the threshold to 0.95.

As we can see in the results above, the accuracy of the message evaluation for the developer is 38%. This means that the developer has written relevant messages. When we look at the developer message manually, we find that the maintainer of this repo has added concise messages such as "fix regex" and "fix dbg," which are not readable and do not fully explain the changes made in the commit.

2. Did the LLM write a precise commit message ??

For this, we will do the same procedure as the previous one. Check the similarity between the diff code and the LLM tags using CodeBERT embeddings. The results above show that the accuracy increased from 38% to 45%. So it means that the comment predictor t5 model generates relevant tags for file edits, but not good enough..

3. To what extent are we able to rectify the messages ??

Again, feed the diff and the rectifier message to code Bert and calculate the similarity. In this case, the accuracy is increased to 65%. This is because Llama 3 from the groq api is a huge general-purpose model, and we have given it a modified file name, developer message, and LLM tags.

We have kept a very high similarity threshold, but if we decrease the threshold to 0.9, then the LLM hit rate and rectifier hit rate becomes 100% whereas the developer hit rate is 99.1%

So, according to the results, the rectifier performs better than the comment predictor t5 model.

Also, the generated message is human-readable compared to that given by the comment predictor.

Take any example from master_commits.csv

For commit hash 7017e94b2f8f06f59bde0409d7c71f864b163cef :

Developer message: fix vflag

LLM tag for files: update version strings | add more support for text converter

Files updated : ['__init__.py', [converter.py](#)]

Rectified Message "Update version strings and enhance text converter support"

References

1. Codebert - Hugging Face ([link](#))
2. Github cheatSheet ([link](#))
3. Groq Docs ([link](#))

Lab 03

Introduction and Concepts

This lab aims to calculate and analyse the code quality metrics. The focus is on structural metrics for measuring Maintainability, cyclomatic complexity, and lines of code for each bug fix before and after the change using radon.

Here is a recap for each term:

1. Maintainability Index

The Maintainability Index is a single score that estimates how easy code is to understand, modify, and support, combining factors like code size, complexity, and the Halstead volume into one number or grade.

2. Cyclomatic Complexity

Cyclomatic Complexity counts the number of independent paths through the code; more branching with if/elif/for/while/switch increases this number and makes the code harder to test and maintain. Formally, it's calculated from a control-flow graph as

$C = E - N + 2P$, where E is edges, N is nodes, and P is the number of connected components, so higher values mean more paths to test and reason about.

Why do we measure before and after the commit ??

Running MI, CC, and LOC before and after a bug fix shows whether the change made the code more straightforward or more complicated to maintain, and by how much. This lets us verify that fixes solve the problem, keep complexity in check, and maintain a healthy maintainability score over time.

Tools and Setup

- Pydriller as in lab 2
- Radon is used to calculate the maintainability index, cyclomatic complexity, and lines of code.
- CodeBERT from Hugging Face
- ScareBLEU

Methodology and Execution

We will use the same code as in lab 2, part C, to create the required dataset. Once done, we will move to part B of the lab to compute and report the baseline descriptive statistics.

Total Number of commits and files.

We can integrate this into the previous code to find how many commits and modified files there are. The separate function to calculate this metric is as follows.

```
def total_commits(repo_url):
    return sum(1 for _ in Repository(repo_url).traverse_commits())

def total_files(repo_url):
    files = set()
    for commit in Repository(repo_url).traverse_commits():
        for m in commit.modified_files:
            if m.new_path:
                files.add(m.new_path)
            elif m.old_path:
                files.add(m.old_path)
    return len(files)
```

Average number of modified files per commit

Calculate the average number of modified files per commit using the simple ratio.

Average = total modified files / commits to summarize change breadth per commit.

```
def avg_mod_files_per_commit(repo_url):
    total_files, total_commits = 0, 0
    for commit in Repository(repo_url).traverse_commits():
        total_commits += 1
        total_files += len(commit.modified_files)
    return total_files / total_commits if total_commits else 0
```

Distribution of fix types llm inference

We will use a counter from standard Python library to track how often the LLM has assigned the given fix tag.

```
def distribution_fix_type_llm(repo_url):
    fix_counter = Counter()

    for commit in Repository(repo_url).traverse_commits():
        for mf in commit.modified_files:
            if mf.diff:
                prediction = safe_infer(mf.diff)
                if prediction:
                    fix_counter[prediction] += 1

    return dict(fix_counter)
```

Most frequently modified filenames/extensions

We can integrate this into the previous code while traversing the repo and the modified file, and we can keep track of how many times files with a given extension are modified.

```
def most_modified_extension(repo_url):
    """Find the most modified file extension in repo history."""
    ext_counter = Counter()
    for commit in Repository(repo_url).traverse_commits():
        for mf in commit.modified_files:
            path = mf.new_path or mf.old_path
            if path and "." in path:
                ext = os.path.splitext(path)[-1]
                ext_counter[ext] += 1
    return ext_counter.most_common(1)[0] if ext_counter else None
```

Putting all the above functions together and running the code

```
def analyze_repo(repo_url, out_file="repo_stats.json", out_format="json"):
    results = {}

    results["total_commits"] = total_commits(repo_url)
    results["total_files"] = total_files(repo_url)
    results["avg_modified_files_per_commit"] = avg_mod_files_per_commit(repo_url)
    results["fix_type_distribution"] = distribution_fix_type_llm(repo_url)
    results["most_modified_extension"] = most_modified_extension(repo_url)

    # Print results nicely
    print("\n=== Repository Analysis ===")
    for k, v in results.items():
        print(f"{k}: {v}")

    # Save results
    if out_format == "json":
        with open(out_file, "w") as f:
            json.dump(results, f, indent=4)
    elif out_format == "csv":
        with open(out_file, "w", newline="") as f:
            writer = csv.writer(f)
            for k, v in results.items():
                writer.writerow([k, v])

    print(f"\nResults saved to {out_file}")
    return results

analyze_repo(repo_url, out_file="b.json", out_format="json")
```

Moving on to part C, we will use radon to calculate the structural metrics.

Below is the code to do as per the assignment, and we will discuss the results later. The code is straightforward; we use the radon function to calculate the necessary metrics.

```
def analyze_code(source):
    if source is None:
        return {"MI": None, "CC": None, "LOC": None}
    try:
        mi = mi_visit(source, True)

        cc_blocks = cc_visit(source)
        cc_total = sum(b.complexity for b in cc_blocks)

        raw = analyze(source)
        loc = raw.loc
        return {"MI": mi, "CC": cc_total, "LOC": loc}
    except Exception:
        return {"MI": None, "CC": None, "LOC": None}

def runradon(repo_url, output_file="c.json"):
    results = []

    for commit in Repository(repo_url).traverse_commits():
        for m in commit.modified_files:
            before = analyze_code(m.source_code_before)
            after = analyze_code(m.source_code)

            entry = {
                "commit": commit.hash,
                "file": m.new_path or m.old_path,
                "MI_Before": before["MI"],
                "MI_After": after["MI"],
                "CC_Before": before["CC"],
                "CC_After": after["CC"],
                "LOC_Before": before["LOC"],
                "LOC_After": after["LOC"],
                "MI_Change": (
                    (after["MI"] or 0) - (before["MI"] or 0)
                    if before["MI"] is not None and after["MI"] is not None
                    else None
                ),
                "CC_Change": (
                    (after["CC"] or 0) - (before["CC"] or 0)
                    if before["CC"] is not None and after["CC"] is not None
                    else None
                ),
                "LOC_Change": (
                    (after["LOC"] or 0) - (before["LOC"] or 0)
                    if before["LOC"] is not None and after["LOC"] is not None
                    else None
                ),
            }
            results.append(entry)

    with open(output_file, "w") as f:
        json.dump(results, f, indent=4)

    print(f"Saved results to {output_file}")
    runradon(repo_url)
```


Now we will calculate the semantic similarity between the source code before and after using CodeBERT. We will also calculate token similarity and add it as a column in the data we created previously.

This is how to set up the model and functions to calculate the similarity using models.

```
# Load CodeBERT once
tokenizer = AutoTokenizer.from_pretrained("microsoft/codebert-base")
model = AutoModel.from_pretrained("microsoft/codebert-base")

def get_codebert_embedding(code):
    if code is None or code.strip() == "":
        return None
    inputs = tokenizer(code, return_tensors="pt", truncation=True, max_length=512)
    with torch.no_grad():
        outputs = model(**inputs)
        emb = outputs.last_hidden_state.mean(dim=1).squeeze().numpy()
    return emb

def semantic_similarity(code1, code2):
    e1 = get_codebert_embedding(code1)
    e2 = get_codebert_embedding(code2)
    if e1 is None or e2 is None:
        return None
    # cosine similarity
    sim = float(torch.nn.functional.cosine_similarity(
        torch.tensor(e1), torch.tensor(e2), dim=0
    ).item())
    return sim

def token_similarity_bleu(code1, code2):
    if code1 is None or code2 is None:
        return None
    ref = code1.split()
    cand = code2.split()
    smoothie = SmoothingFunction().method1
    return sentence_bleu([ref], cand, smoothing_function=smoothie)
```

Now we need to add two more fields in the previous loop as follows:

```

entry = {
    "commit": commit.hash,
    "file": m.new_path or m.old_path,
    "MI_Before": before["MI"],
    "MI_After": after["MI"],
    "CC_Before": before["CC"],
    "CC_After": after["CC"],
    "LOC_Before": before["LOC"],
    "LOC_After": after["LOC"],
    "MI_Change": (
        (after["MI"] or 0) - (before["MI"] or 0)
        if before["MI"] is not None and after["MI"] is not None
        else None
    ),
    "CC_Change": (
        (after["CC"] or 0) - (before["CC"] or 0)
        if before["CC"] is not None and after["CC"] is not None
        else None
    ),
    "LOC_Change": (
        (after["LOC"] or 0) - (before["LOC"] or 0)
        if before["LOC"] is not None and after["LOC"] is not None
        else None
    ),
    # New Change Magnitude Metrics
    "Semantic_Similarity": semantic_similarity(before_code, after_code),
    "Token_Similarity": token_similarity_bleu(before_code, after_code),
}
results.append(entry)

```

So, now moving on to the final step, we have to classify a commit as major or minor based on semantic similarity and token similarity individually.

For that, we need to add two other checks in our central `lo, o, p`, which are as follows.

```

# final addition
"Semantic_Class": (
    "Minor" if semantic is not None and semantic >= semantic_threshold else "Major"
),
"Token_Class": (
    "Minor" if token is not None and token >= token_threshold else "Major"
),

```

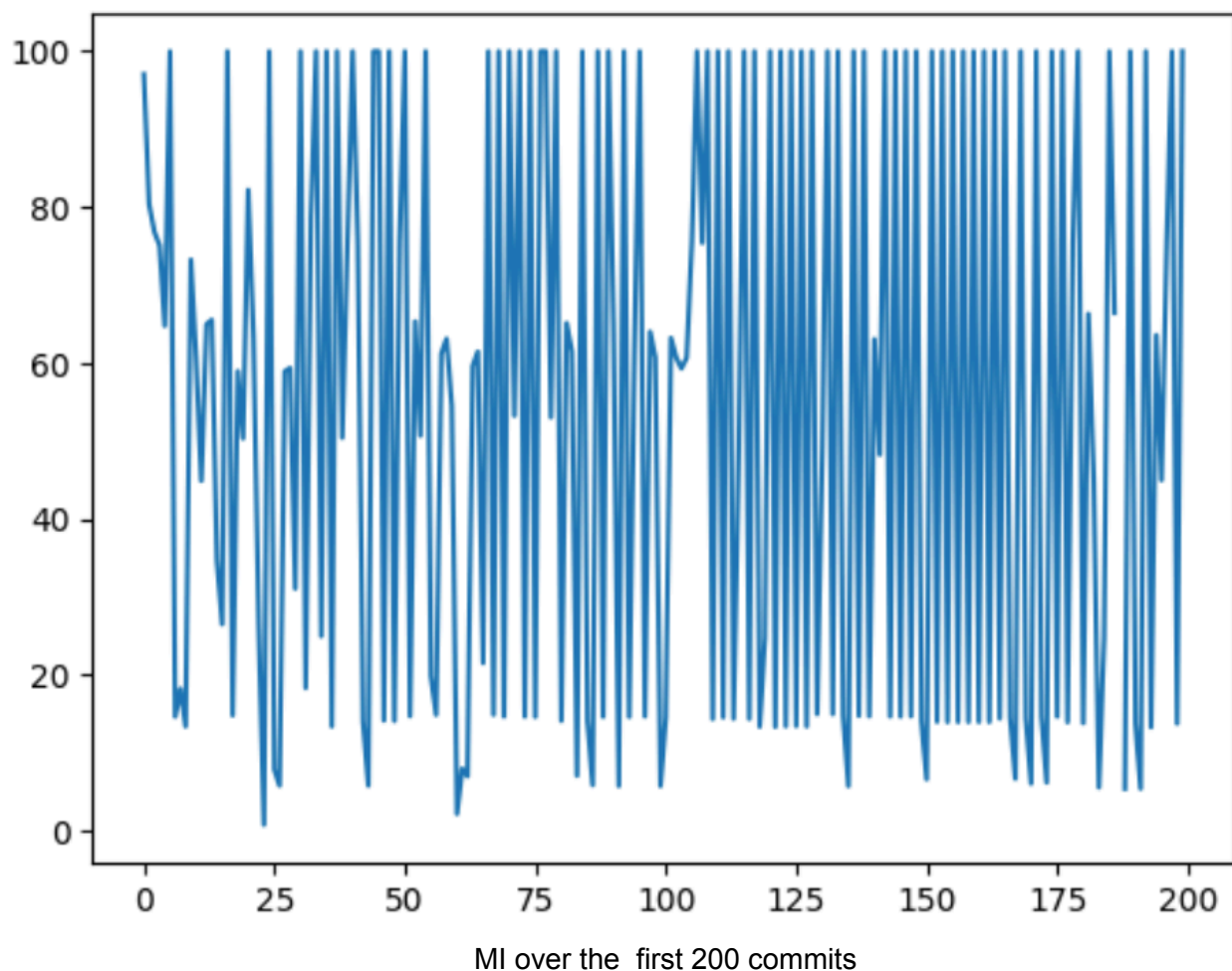
Result Analysis and Discussion

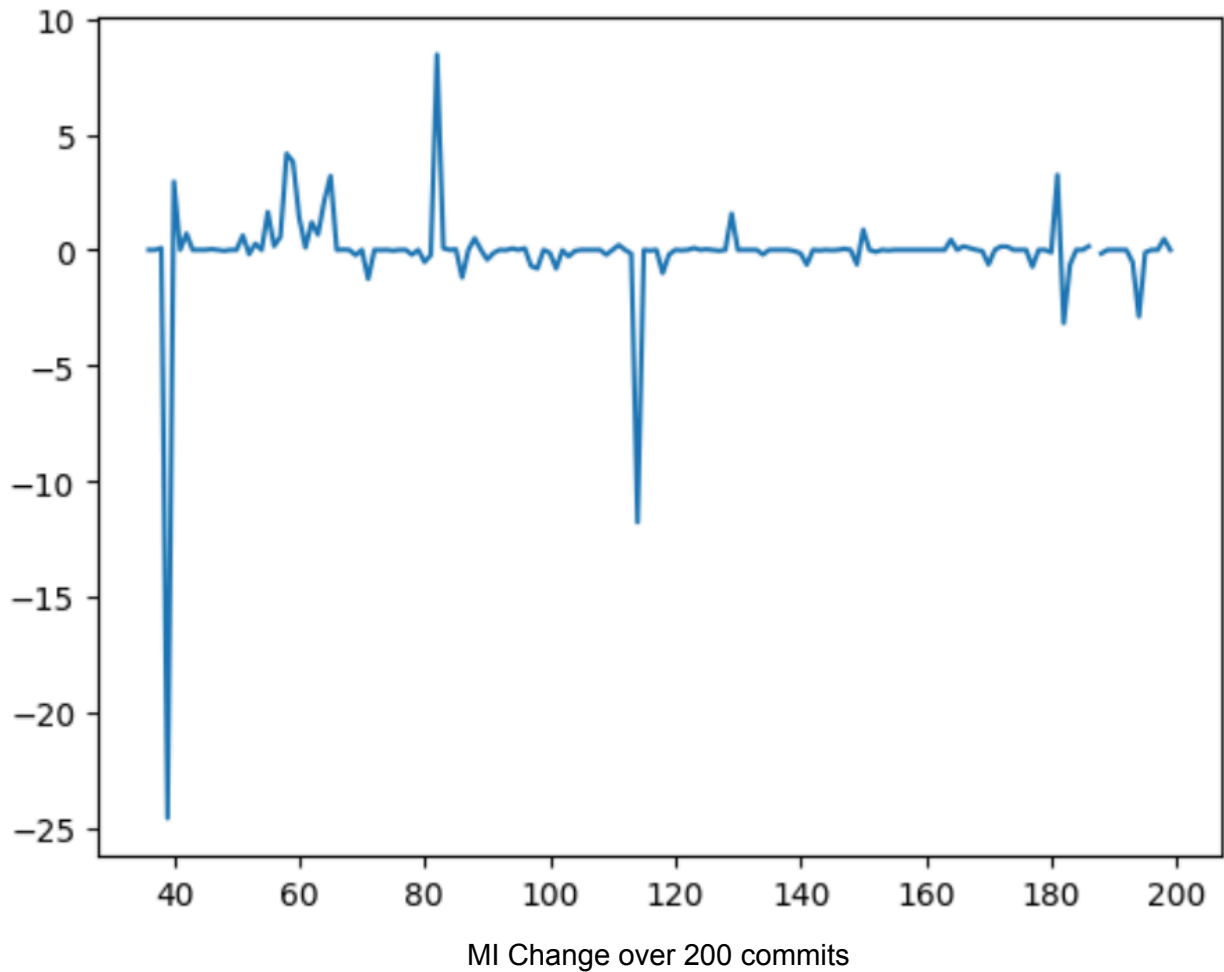
Now we have created the required metadata for our repository. We can see which files are edited the most, which commits are major commits, and which are minor in the context of semantic and token similarity. We also have all the structural metrics that show the project's complexity.

We will analyze the first 200 commits to see the metrics' trends.

Maintainability Index

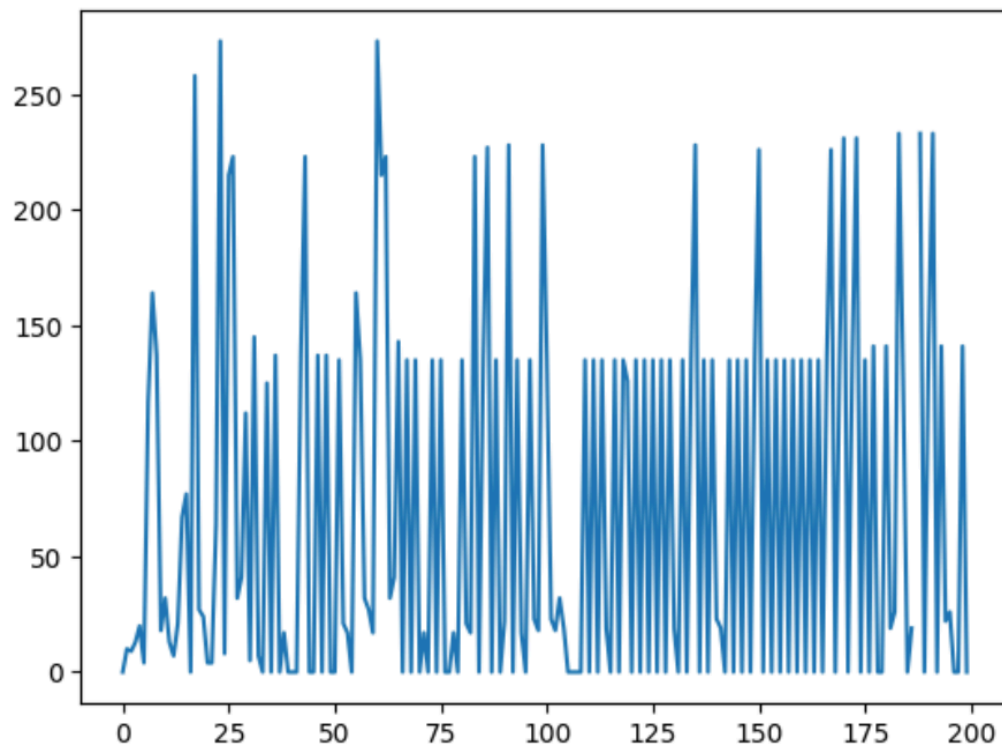
The maintainability index changes over time. As shown in the MI_change graph, there is not much change except for a few times. Sometimes, a sudden drop suggests that the commit is not good, and someone has pulled unorganized code. Most of the time, MI is above 90. This helps us to find the unorganized commits and restructure the code.



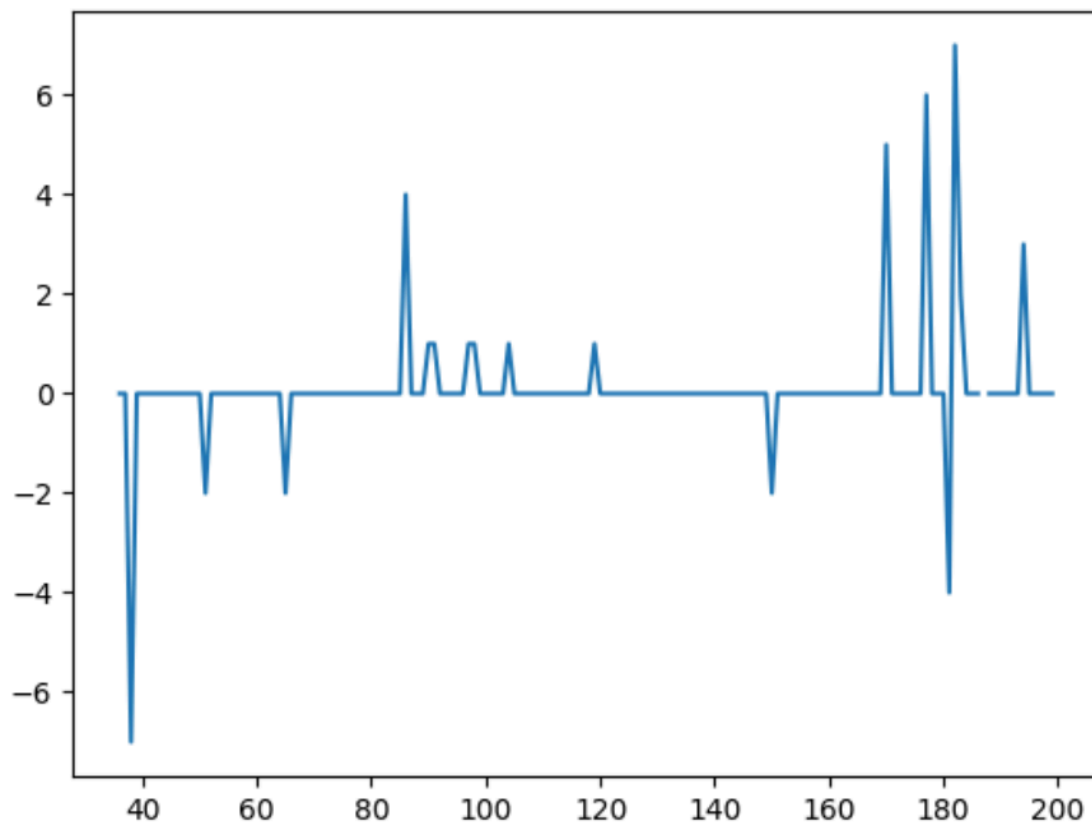


Cyclomatic complexity

We have a similar analysis to that of MI. The cyclomatic complexity must be low for the code to be readable and easy to debug. In the change graph, we can see some dips indicating that the code quality added to that commit is poor, and we need to fix that.



Cyclomatic complexity over the first 200 commits

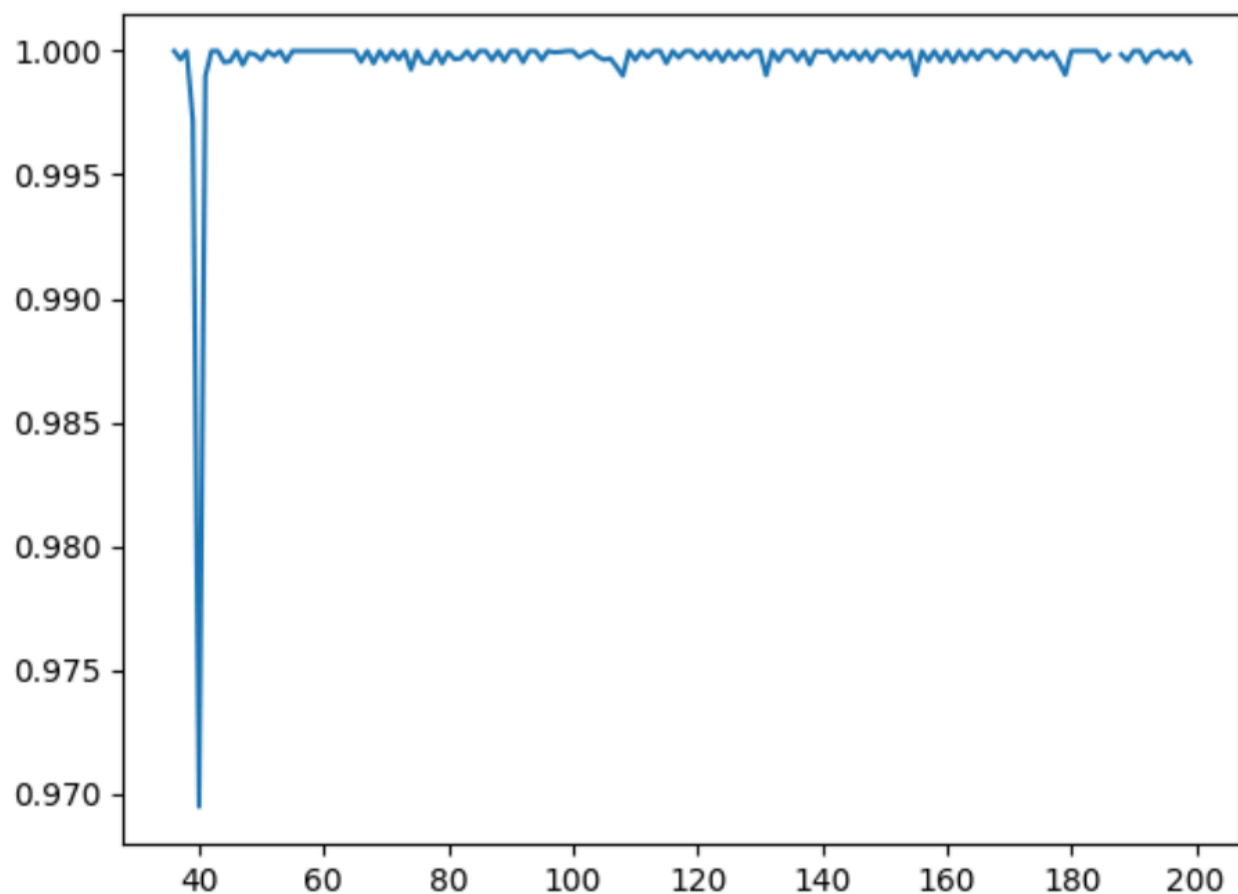


Cyclomatic Complexity change over the first 200 commits

Semantic Similarity

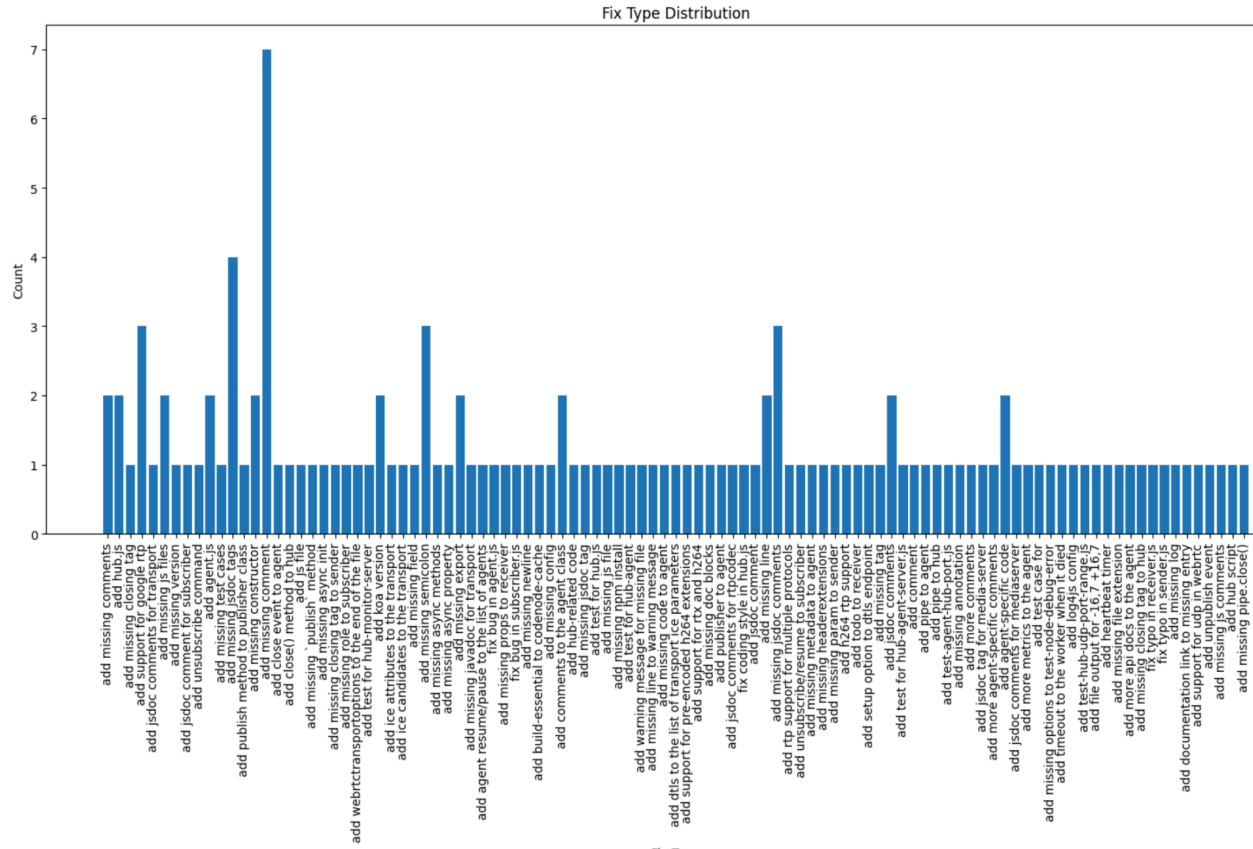
Now we analyse the semantic similarity between the source code before and after the commit. Most of the time, the similarity is above 0.95, suggesting that no significant change has been made to the code base. There were not many essential changes in the selected repo after the code had been set up. However, in some repos, we can expect very low similarity where the maintainers have decided to make significant changes to the code base or to restructure it. Also, in the initial commits, we take the similarity as zero as new files are added that would contribute significant changes.

The graph below shows the semantic similarity for the first 200 commits. We can see a few significant dips in the similarity score due to substantial changes in the code base.



Semantic Similarity before and after the commit is done.

Distribution of bug fix - llm tags



According to the histogram, we can see that most of the time, the fix is done by adding comments to the code. Another major bug fix is adding support for Google RTP and adding agent-specific code.

This histogram depicts the most and least bugs and fixes done to the codebase, which is very helpful for future decision-making and checking the contributors if it is a large open source project.

References

1. Lecture Slides
2. Radon Documentation([link](#))
3. Github cheatSheet ([link](#))

Lab 04

Introduction and Concepts

This lab assignment analyzes different diff algorithms, especially Myers and histogram diff. We pose questions like whether they are the same for most of the files and which is better, and we store the metadata in a CSV file.

Tools and Setup

- Pydriller: For traversing the repositories
- Seart: for searching repositories

Methodology and Execution

First, we will need to select three repositories, and for that, we will keep the size of the repositories small because it takes a lot of time to run. So our criteria are

- At least 1000 commits
- At least 10000 stars
- Any language would work
- We will hand-pick the famous projects from the sorted list

The screenshot shows the SEART search interface. The 'General' section has a search bar with 'python' entered. The 'History and Activity' section has 'Number of Commits' set to 1000. The 'Popularity Filters' section has 'Number of Stars' set to 10000. The 'Additional Filters' section has 'Exclude Forks' checked. The 'Search' button is at the bottom.

We are using the following repositories.

- PDFMathTranslate
- Flash-attention

- Ggml

How to get Myers diff ??

We will run the system. Subprocess and execute the git command to get the Myers diff for two commits. We will get the hash of the commits by pydriller while traversing the repository.

```
def myers_diff(repo_path: str, commit_hash: str, parent_hash: str, file_path: str) -> str:
    try:
        result = subprocess.run(
            ["git", "-C", repo_path, "diff", parent_hash, commit_hash, "--", file_path],
            stdout=subprocess.PIPE,
            stderr=subprocess.PIPE,
            text=True,
            check=True,
            errors="replace"
        )
        lines = result.stdout.splitlines()
        trimmed = []

        keep = False
        for line in lines:
            if line.startswith("@@"):
                keep = True
            if keep:
                trimmed.append(line)

        return "\n".join(trimmed).strip()
    except subprocess.CalledProcessError as e:
        return f"Error getting diff: {e.stderr.strip()}"
```

How to get histogram diff ??

Pydriller uses histogram diff by default, and we will use commit.diff simply.

How do I get the type of files ??

To find the file type, we will use the path to determine whether it is a source or test file. The function that returns the file type based on the path is

```
def find_filetype(path: str) -> str:
    """Categorize file type based on its path."""
    if not path:
        return "other"
    path_lower = path.lower()
    if "test" in path_lower or "tst" in path_lower:
        return "test"
    if "src" in path_lower or "source" in path_lower:
        return "source"
    if "readme" in path_lower:
        return "readme"
    if "license" in path_lower:
        return "license"
    return "other"
```

Now we have all our tools ready. We will build upon the code from lab 2 and add the necessary columns to the data using the functions we discussed.

```
def collect_commit_data(repo_path: str):
    """Traverse repository commits and collect diff information."""
    max_commits = int(1e7)
    commit_data = []
    global commits_counter
    total_commits = 0

    for commit in Repository(repo_path, histogram_diff=False).traverse_commits():
        total_commits += 1
        commits_counter += 1

        if max_commits == 0:
            break
        max_commits -= 1

        if not commit.parents:
            continue
        parent_sha = commit.parents[0]

        for file in commit.modified_files:
            old_path = file.old_path
            new_path = file.new_path

            histdiff = file.diff.strip()
            myersdiff = myers_diff(
                repo_path,
                commit.hash,
                parent_sha,
                new_path or old_path
            )

            if histdiff.strip() == "" or myersdiff.strip() == "":
                continue

            isequal = histdiff == myersdiff
            category = find_filetype(new_path or old_path)

            commit_data.append({
                "old_path": old_path,
                "new_path": new_path,
                "commit SHA": commit.hash,
                "parent commit SHA": parent_sha,
                "commit message": commit.msg,
                "diff_hist": histdiff,
                "diff_myers": myersdiff,
                "Discrepancy": not isequal,
                "Category": category
            })

    return commit_data, total_commits
```

And finally, we will wrap this function along with the functionality to plot the histogram of total discrepancy so that calling it for three repos becomes easier.

```
def run_on_url(repo_url):
    commit_data, total_commits = collect_commit_data(repo_url)

    df = pd.DataFrame(commit_data)
    csv_path = f"{repo_url}.csv"
    df.to_csv(csv_path, index=False)

    print(f"\nTraversed {total_commits} commits.")
    print(f"Saved {len(df)} file changes to {csv_path}")

    if "Discrepancy" in df.columns and not df.empty:
        mismatches = df[df["Discrepancy"] == True]
        print(f"\nFound {len(mismatches)} discrepancies.")

        if not mismatches.empty:
            stats = mismatches["Category"].value_counts()
            print("\n# Mismatches by category:")
            print(stats)
        else:
            print("\nNo data collected or 'Discrepancy' column not found.")

    import matplotlib.pyplot as plt
    plt.figure(figsize=(8,5))
    stats.plot(kind="bar", color="skyblue", edgecolor="black")
    plt.title("Mismatches by File Category")
    plt.ylabel("Count of Mismatches")
    plt.xlabel("File Category")
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.savefig("mismatch_stats.png")
    plt.show()

run_on_url("PDFMathTranslate")
run_on_url("flash-attention")
run_on_url("ggml")
```

How would we determine which algorithm performs better ??

Examining how each one works and its strengths is necessary to determine which algorithm performs better.

Myers Algorithm relies on traversing an edit graph, calculating the shortest possible sequence of edits needed to turn one code snippet into another. Its time complexity is $O(ND)$, where N is the sequence length and D the edit distance, and it uses $O(n+m)$ space. While Myers is precise, it tends to be slower than the histogram approach when working with large codebases.

The Histogram Algorithm is based on counting how often each line appears within the code. This method generally runs in linear time and is typically much faster than Myers, making it a good choice for projects where speed matters and not the exact diff.

There is a tradeoff between precision and time. So, there is no particular winner between these algorithms, but we can determine which is better for a given use case.

When comparing both algorithms on a repository, two key benchmark metrics should be analyzed:

1. Execution time
2. The minimum number of differing lines in the output

Focusing on these metrics makes it easier to objectively compare their effectiveness and choose the most suitable algorithm for a particular use case.

Result Analysis and Discussion

NOTE: By discrepancy, we refer to the mismatch between Myers and the histogram diff

PDFMathTranslate

There is no separate folder of source files, so they are classified as other files. There are not many discrepancies in this repository, meaning that most of the time the changes in the code are made so that they do not conflict much, and that is why Myers and histogram are giving the same diffs.

```
Traversed 1067 commits.  
Saved 1607 file changes to PDFMathTranslate.csv
```

```
Found 13 discrepancies.
```

```
# Mismatches by category:
```

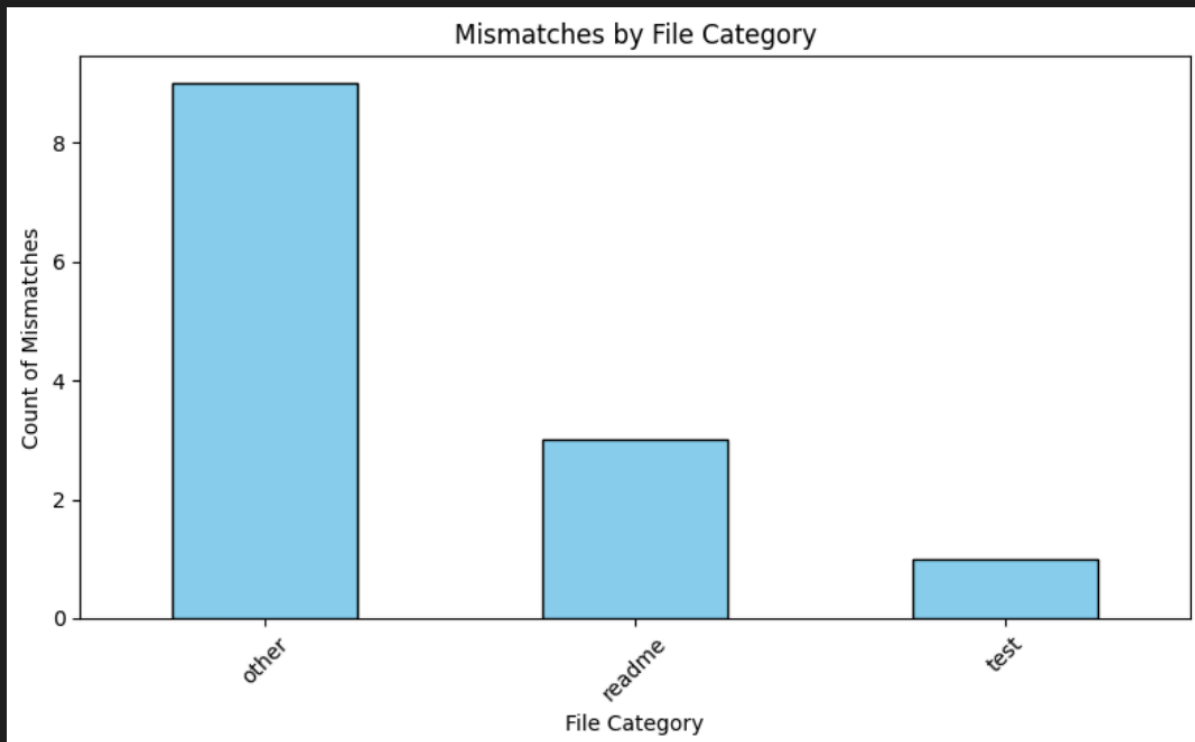
```
Category
```

```
other      9
```

```
readme     3
```

```
test       1
```

```
Name: count, dtype: int64
```



Flash-attention

In this repository, most discrepancies are in other files, not the source and test files. This suggests that most of the changes are done in different folders. Flash attention is a model based on a research paper, and there are folders such as training, where the main training is done, and in folders named benchmark, benchmarking of the model is done. Also, it is based on a research paper that leads to fewer changes to the model architecture, put under the src and tests, compared to the other training and benchmarking parts. Therefore, in this case, most discrepancies lie outside tests and the src folder.

```
Traversed 1009 commits.  
Saved 5513 file changes to flash-attention.csv
```

```
Found 33 discrepancies.
```

```
# Mismatches by category:
```

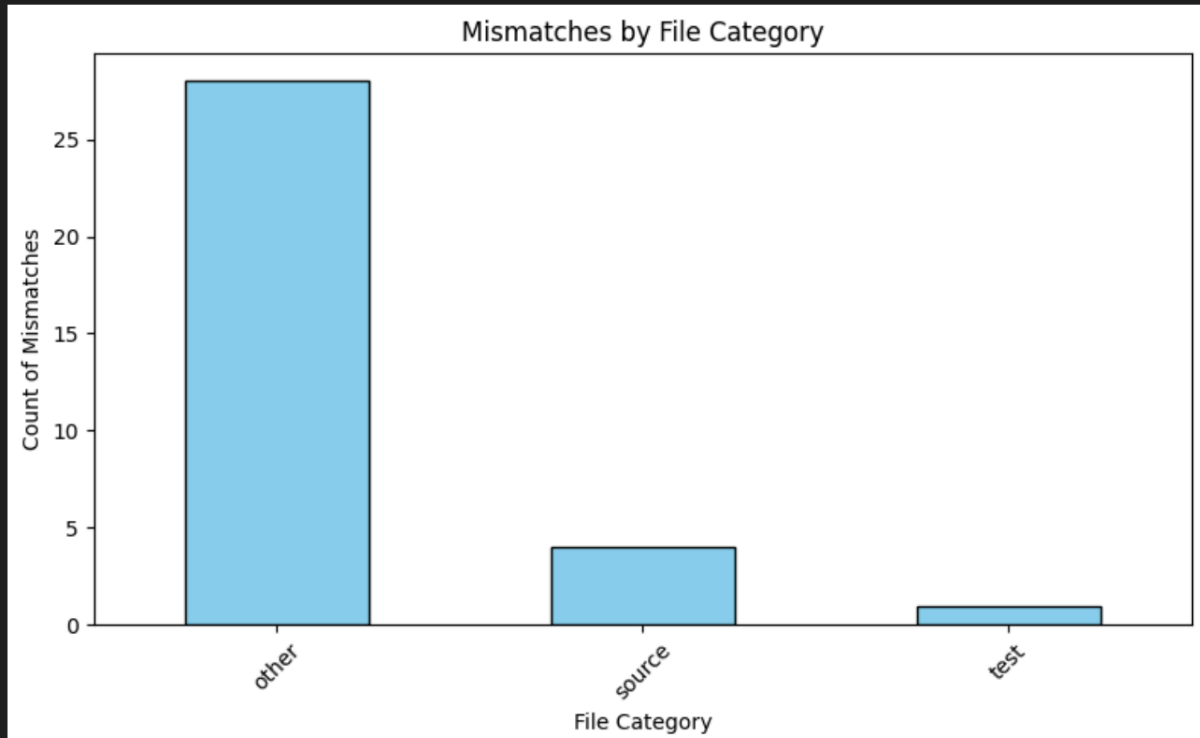
```
Category
```

```
other      28
```

```
source      4
```

```
test        1
```

```
Name: count, dtype: int64
```



Ggml

Ggml is a c/c++ library for machine learning, like PyTorch. The code in this repository is well organized, and there are README, license, src, and test files. Most of the discrepancies lie in the test and other files.

```
Traversed 2472 commits.  
Saved 7796 file changes to ggml.csv
```

```
Found 31 discrepancies.
```

```
# Mismatches by category:
```

```
Category
```

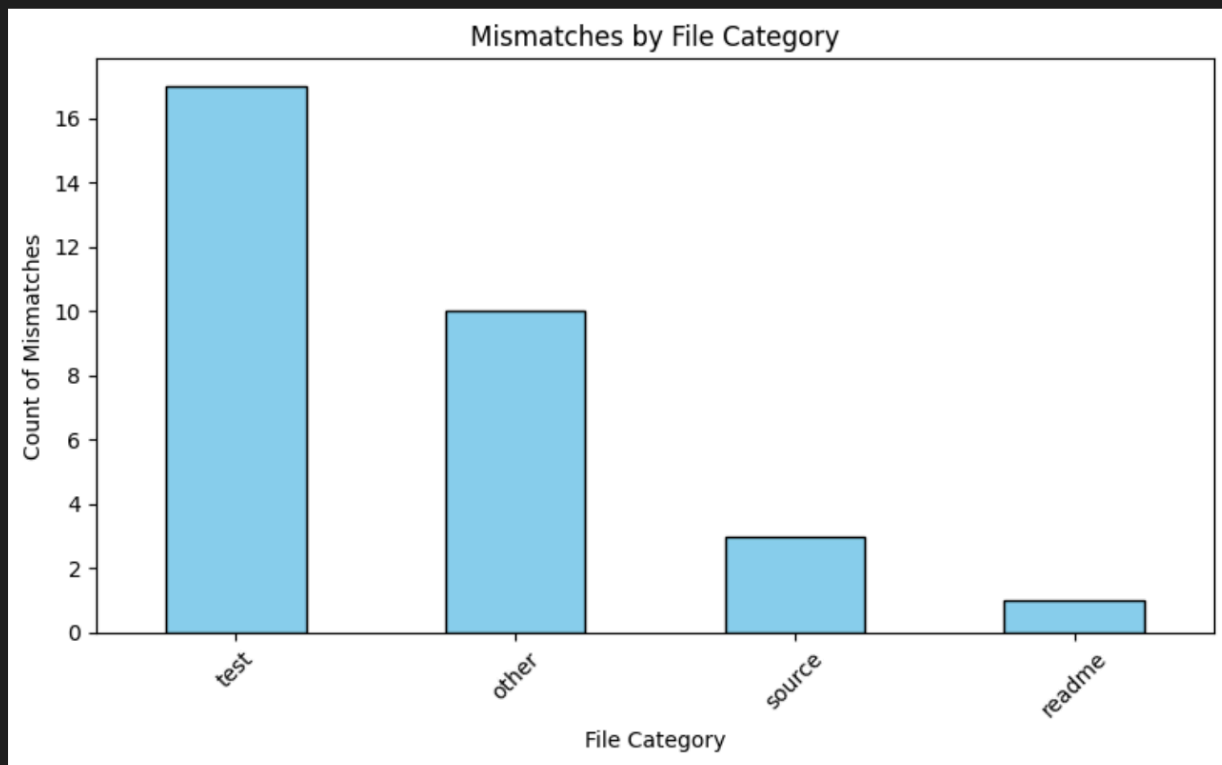
```
test      17
```

```
other     10
```

```
source     3
```

```
readme     1
```

```
Name: count, dtype: int64
```



References

1. pydriller documentation ([link](#))
2. Github cheatSheet ([link](#))
3. Subprocess documentation ([link](#))